04-Panduan-Penggunaan-AI

Contents

04 · PANDUAN PENGGUNAAN AI — ZippyVerse 1
1 · Tools yang Dipakai 1
2 · Alur Kerja 2
3 · Cara Menulis Prompt yang Efektif 3
4 · Cara Memeriksa Hasil Kerja AI 6

04 · PANDUAN PENGGUNAAN AI — ZippyVerse

Panduan ini menjelaskan **bagaimana** proyek ini dibangun bersama AI, supaya iterasi berikutnya tidak mengulang kesalahan yang sama.

---

1 · Tools yang Dipakai

1.1 · Lingkungan kerja

Alat	Peran
OpenCode	Agen yang menjalankan perintah, membaca/menulis berkas, dan berinteraksi dengan peramban
Model utama — `mimo-v2.6-flash-free`	Pekerjaan harian: membaca kode, menyunting, menjalankan tes
`space-bunny-free`	7 baris pesan
`claude-opus-5.5`	3 baris pesan — dipakai pada perumusan ide awal
Git	Pencatatan setiap perubahan
Vercel	Hosting dan produksi
Supabase	Penyimpanan terkelola

Rekam jejak nyata dari metadata sesi (80 baris pesan):

Model	Jumlah
`mimo-v2.6-flash-free`	62
`space-bunny-free`	7
`claude-opus-5.5`	3
tanpa keterangan model	8 (5 prompt pembuka + 3 pesan pemicu subagen)

Penyedia = **OpenCode**. Menyebut "OpenCode · MiMo" betul tetapi perlu dilengkapi: tiga model benar-benar dipakai.

1.2 · Penggunaan subagen

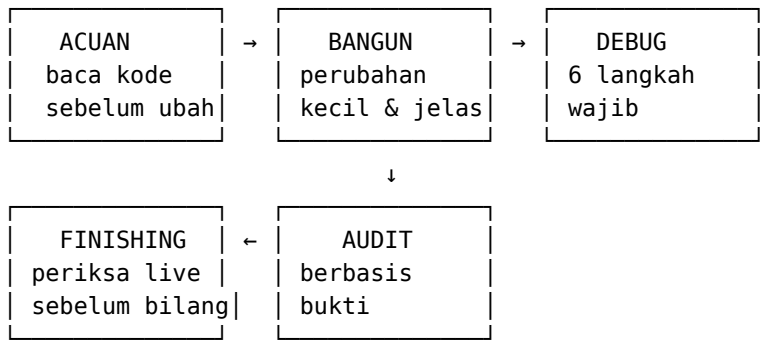
Tiga audit dijalankan **paralel sebagai subagen** pada 13 Okt 13:13 WIB:

1. *Security and privacy audit*
2. *Accessibility and SEO audit*
3. *Code quality and content audit*

Ketiganya **ditulis oleh AI**, bukan oleh manusia, sehingga tidak dihitung sebagai prompt pengguna dan diletakkan terpisah dalam Lampiran B log.

---

2 · Alur Kerja



Tahap 1 · ACUAN

Baca dulu, ubah kemudian. `AGENTS.md` adalah rujukan utama; `README.md` untuk cara menjalankan; log prompt untuk memahami mengapa keputusan tertentu diambil.

Aturan: temukan pola yang sudah dipakai di berkas sejenis, lalu ikuti — jangan memperkenalkan gaya baru.

Tahap 2 · BANGUN

- Satu tujuan per perubahan.
- Perubahan kecil dan mudah dibedakan.
- **Jangan memperbaiki sesuatu yang belum diminta izinnya** — laporkan temuan lebih dulu.

Tahap 3 · DEBUG — enam langkah yang diwajibkan

Prompt penggeraknya (Prompt #70, 15 Okt 12:45 WIB) memutus kebiasaan menebak:

#	Langkah	Yang diminta
1	**REPRODUKSI**	Jelaskan langkah memicu bug dan gejala persisnya
2	**DIAGNOSA**	Telusuri kode terkait; jelaskan **KENAPA** terjadi, bukan hanya "perbaiki X"
3	**PERBAIKAN MINIMAL**	Fix paling kecil dan aman; dilarang menulis ulang besar-besaran
4	**VERIFIKASI**	Jalankan build/tes; pastikan bug hilang dan tidak ada error baru
5	**COMMIT**	Pesan deskriptif: `fix: [fitur] - [masalah] ([penyebab])`
6	**CATATAN JURNAL**	Gejala, akar masalah, perbaikan ⟶ `JURNAL-DEBUGGING.md`

Mengapa aturan ini lahir: sebelumnya bug diperbaiki sekenanya. Sejak aturan ini, sebagian perbaikan menyentuh `tests/` sekaligus — bug diperbaiki **dengan uji regresi**, bukan ditambal.

Tahap 4 · AUDIT

Prompt audit (Prompt #34, 13 Okt 13:10 WIB, 21.767 karakter) menetapkan aturan ketat:

- **Dilarang** menganggap fitur berjalan hanya karena elemen UI-nya ada.
- **Dilarang** mengarang hasil tes, ukuran performa, atau Core Web Vitals.
- **Wajib** membedakan temuan terverifikasi, analisis statis, dan dugaan.
- **Dilarang** membocorkan rahasia ke laporan — cukup sebutkan lokasinya.
- Mode bawaan **`AUDIT ONLY`** — tidak boleh menyentuh kode pada putaran pertama.

Tahap 5 · FINISHING

Sebelum menyatakan selesai:

```
npm test        # 184 pengujian
npm run lint
npm run build
```

Lalu **periksa di situs yang benar-benar hidup di internet.** Ini tahap yang paling sering bohong: sebuah website bisa lolos build, lint, dan 184 pengujian **tetap rusak di internet**.

---

3 · Cara Menulis Prompt yang Efektif

Disaring dari prompt yang terbentuk berhasil selama pengerjaan.

3.1 · Enam pola yang terbukti bekerja

#	Pola	Contoh dari proyek ini
1	**Sebutkan masalah nyata, bukan solusi yang sudah kamu tentukan**	*"anak anak sekarang rata rata banyak yang lebih dominan menonton video di sosmed … jadi hal apa yang bikin anak anak tertarik"* — AI diizinkan mencari sendiri
2	**Perintahkan cara berpikirnya, bukan hanya hasilnya**	Enam langkah debug: REPRODUKSI → DIAGNOSA → … → CATATAN JURNAL
3	**Larang asumsi secara eksplisit**	*"Jangan mengarang. Kalau tidak bisa diverifikasi, tulis BELUM JELAS."*
4	**Minta bukti per poin, bukan klaim kelar**	Verifikasi deploy: tiap poin diberi status **OK/ERROR + bukti**
5	**Beri batas yang jelas**	*"tanpa merubah fitur fungsi"*, *"tanpa mengubah fitur lain"*, *"maks 5 pesan/hari"*
6	**Minta laporan tiap langkah, bukan hanya akhir**	*"Laporkan hasil tiap langkah"*

3.2 · Kerangka prompt debug

```
Ada bug di [fitur]. Kerjakan dengan alur berikut (jangan langsung menebak):
1. REPRODUKSI: langkah memicu bug + gejala persisnya.
2. DIAGNOSA: telusuri kode (file, fungsi, state, data flow), jelaskan KENAPA.
3. PERBAIKAN MINIMAL: fix paling kecil, tanpa mengubah fitur lain.
4. VERIFIKASI: jalankan build/tes, pastikan tidak ada error baru.
5. COMMIT: "fix: [fitur] - [masalah] ([penyebab])"
6. CATATAN JURNAL: simpan ke [nama berkas].
Laporkan hasil tiap langkah.
```

3.3 · Kerangka prompt audit

```
Periksa [lingkup]. Aturan:
- Jangan menganggap fitur berjalan hanya karena UI-nya ada.
- Jangan mengarang hasil tes atau ukuran performa.
- Bedakan: terverifikasi / analisis statis / dugaan.
- Jangan menyimpan rahasia di laporan.
- AUDIT ONLY — jangan ubah kode sebelum persetujuan.
Kelompokkan temuan: P0 kritis · P1 tinggi · P2 sedang · P3 rendah.
```

3.4 · Kesalahan yang sebaiknya dihindari

Kesalahan	Akibat	Perbaikan
Menyuruh "perbaiki" tanpa menyuruh mereproduksi	Bug ditambal, muncul lagi	Wajibkan REPRODUKSI lebih dulu
Membiarkan AI menyimpulkan sendiri batasannya	Perubahan melebar, tak terduga	Tulis batas: "tanpa mengubah fitur lain"
Menyatakan selesai dari hasil build saja	Rusak di internet tanpa terasa	Selalu verifikasi di URL live
Mengandalkan angka dari ingatan	Dokumen saling bertentangan	Minta perintah yang bisa dijalankan ulang
Menyalin rahasia ke prompt/dokumen publik	Rahasia bocor	Ganti dengan penanda `[RAHASIA DIREDAM: …]`

3.5 · Aturan menyangkut rahasia

Empat rahasia pernah tercatat dalam percakapan dan kini tersimpan sebagai penanda di log publik. Keempatnya **sudah terhitung bocor dan wajib dibuat baru**:

Rahasia	Cara mengganti
`GEMINI_API_KEY`	Google AI Studio → Create API key
Kata sandi koneksi database	Supabase → Settings → Database → Reset credentials
Kata sandi akun pengelola	Supabase → Authentication → Users → ubah password
`SUPABASE_SERVICE_ROLE_KEY`	Supabase → Settings → API → tugas layanan

Kata sandi akun dan kata sandi database saat ini **sama nilainya**, jadi keduanya selesai dengan sekali ganti.

Aturan tetap: rahasia tidak pernah masuk berkas, halaman, log, atau laporan. Cukup sebutkan *lokasinya*.

---

4 · Cara Memeriksa Hasil Kerja AI

#	Pemeriksaan	Perintah
1	Pengujian	`npm test` — 184 pengujian harus lulus
2	Lint	`npm run lint`
3	Build	`npm run build`
4	Riwayat	`git log --oneline` — pesan harus menjelaskan **apa dan mengapa**
5	Angka di dokumen	Jalankan ulang perintah yang disebut; jangan percaya ingatan
6	Situs hidup	Buka URL live, jangan hanya percaya hasil build

Prinsipnya: bedakan yang **sudah dipastikan** dari yang **masih dugaan**. Hasil pengamatan langsung ditulis berbeda dari perkiraan tanpa bukti.

Disusun 15 Okt 2026. Bagian 1.2 dan 1.1 berasal dari metadata sesi OpenCode; bagian 2 dan 3 dari prompt asli yang tercatat di `LOG-PROMPT-MENTAH.md`.